

Progetto Object Oriented Programming “Orcs Arena” - Documentazione

Studente: Thomas Andrea Porcari

Matricola: 24254

Corso: Ingegneria Informatica per la Transizione Digitale

Contesto del Software e Destinatari

- **Obbiettivo:** Realizzare un gioco di ruolo a turni su riga di comando per testare i concetti di programmazione a oggetti (classi astratte, incapsulamento, design pattern e gestione dei file). Il gioco fa affrontare a una squadra di 1 o 2 giocatori una serie di nemici con difficoltà crescente.
- **Audience:** Utenti interessati ad un gioco testuale a turni con difficoltà crescente.

Esempio di Funzionamento

1. All'avvio il menu permette di iniziare una nuova partita, caricarne una vecchia dal file .txt di salvataggio, vedere la Hall of Fame dal file .bin oppure uscire.
2. Se si sceglie una nuova partita, si sceglie se si vuole giocare da soli o in coppia e si scelgono nomi e classi. Le classi vengono create dalla factory “creatorepersonaggio”. (Design Pattern Creazionale)
3. Inizia il round 1 con il primo orco, i giocatori possono attaccare, curarsi, usare oggetti o salvare ed uscire.
4. Quando un nemico muore, i giocatori salgono di livello, recuperano la vita e ottengono un oggetto casuale.
5. Se tutti i giocatori vanno a 0 HP, la partita finisce; il punteggio viene calcolato e salvato in automatico nel file binario della classifica. Lo stesso avviene se si superano tutti gli stage.

Struttura del Codice:

Modulo main.py

Game: classe principale con players, inventory, stage e num_players.

Metodi: nuovogioco(), caricagioco(), salvagioco(), choose_item(), totalpartykill(), menu_combat() e menu_principale().

Modulo creatore.py (**Pattern Factory**)

creatorepersonaggio: Metodo statico create(classe, name) che riceve una stringa e restituisce la giusta istanza (Mage o Warrior), sollevando un errore se la classe scelta non esiste. Utile nel concetto di scalabilità in quanto si gestisce la logica della creazione del personaggio riguardo la sua classe fuori dal main

Modulo characters.py

personaggi(ABC): Classe astratta generica per i personaggi giocabili.

Attributi protetti: _name (nome), _level (livello), _max_hp (vita massima), _hp (vita attuale).
Vengono letti tramite @property per non modificarli dall'esterno.

Metodi: ancoravivo() (controlla se la vita > 0), take_damage (riduce la vita del giocatore senza andare sotto zero), heal (cura la vita del giocatore), full_vita() (cura tutta la vita del giocatore), levelup() (aumenta livello e vita max), status() (stampa la barra della vita e della risorsa).

Metodi astratti: nome_risorsa, valore_risorsa, max_risorsa, refill_risorsa, attack, curarsi, usaoggetto (implementati dai figli).

Mage(personaggi): Gestisce il mago con il Mana.

Attributi: mana e max_mana.

Metodi: attack (consuma 3 mana e fa danno magico), curarsi (consuma 4 mana e cura), usaoggetto (applica gli oggetti consumabili).

Warrior(personaggi): Gestisce il guerriero con la Stamina.

Attributi: stamina e max_stamina.

Metodi: attack (sceglie potenza 1, 2 o 3 e scala stamina proporzionalmente), curarsi (spende stamina per curarsi), usaoggetto (usa oggetti nell'inventario).

Modulo pozioni.py (**Pattern Decorator**)

ScudoMagico: avvolge il personaggio aggiungendo uno scudo temporaneo senza cambiare la sua classe originale.

Attributi: target (il personaggio decorato), turni (quanti turni dura il buff).

Metodi: take_damage (dimezza il danno e lo passa al target), scala_turno (riduce di 1 il contatore e dice se il buff è ancora attivo). Tutti gli altri metodi li rimanda direttamente al target.

In futuro si potrebbero implementare altre pozioni che vanno ad incidere su altre caratteristiche per un numero fissato di turni

Modulo strategianemica.py (**Pattern Strategy**)

Strategia(ABC): Interfaccia per l'algoritmo di selezione del bersaglio.

Metodi: scegli_bersaglio (metodo astratto).

attaccarandom: Prende un giocatore a caso tra quelli vivi.

conmenovita: Cerca e restituisce il giocatore con meno HP.

conpiuvita: Cerca e restituisce il giocatore con più HP.

E' chiaramente possibile integrare altre strategie che utilizzino altri criteri per determinare quale giocatore attaccare, queste 3 fungono da esempio

Modulo nemici.py

Enemy: Classe per il mostro avversario.

Attributi: name, level, max_hp, hp, base_damage, strategia.

Metodi: ancoravivo, take_damage, attack e barre_di_stato.

summona_nemico: Funzione che calcola le statistiche dell'orco in base al round e gli assegna la strategia corretta.

Modulo hall_of_fame.py (Persistenza su File Binario)

HallOfFame: Gestisce la classifica dei migliori 10 punteggi leggendo e scrivendo con rb e wb.

Parametri: file di 20 byte (16 byte per il nome, 2 byte per lo stage, 2 byte per il punteggio).

Metodi statici:

aggiungi_punteggio(nomi, stage, punti): aggiunge il record, ordina in modo decrescente, tiene i primi 10 e salva usando open("wb") con int.to_bytes().

leggi_classifica: apre con open("rb"), legge a blocchi di 20 byte con read(20) e decodifica i valori.

mostra(): stampa a video la top 10.

Modulo saveload.py (Persistenza su File di Testo)

saveload: Salva e riprende la partita da salvataggio.txt.

Metodi statici: save (scrive le righe chiave=valore con stage, nomi, classi e inventario) e load (legge il file, ricostruisce i giocatori e ripristina il livello giusto chiamando levelup).

Si è scelto appositamente di far ricominciare lo stage una volta caricata la partita così da evitare eventuali exploit